Experiment 10

Implementation of Hyperledger Fabric Network and Asset Management using Chaincode

Aim

To implement a Hyperledger Fabric network, deploy chaincode, and perform asset creation, querying, and transfer operations.

Tasks Performed

1. Prepared the environment (Docker, Git, Node.js, npm, `jq`, `curl`) and cloned `hyperledger/fabric-samples`.
2. Installed the Fabric binaries and Docker images with `install-fabric.sh`, and set `PATH` and `FABRIC_CFG_PATH` (`peer version`).
3. Started the test network and created the channel: `./network.sh up createChannel`.
4. Deployed the `asset-transfer-basic` JavaScript chaincode: `./network.sh deployCC`.
5. Set the Org1 peer environment, initialised the ledger (`InitLedger`) and queried all assets and `asset5`.
6. Created a new asset (`CreateAsset`) and transferred it (`TransferAsset`), checking the ledger after each step.
7. Stopped the network with `./network.sh down`.

Theory

Hyperledger Fabric is a permissioned, enterprise blockchain framework: every participant has an identity issued by a certificate authority and belongs to an organization, unlike public chains where anyone can join. Its main components are:

- **Peers** — hold a copy of the ledger and execute chaincode to endorse transactions. The test network has `peer0.org1.example.com` (Org1MSP) and `peer0.org2.example.com` (Org2MSP).
- **Orderer** — orders endorsed transactions into blocks (`orderer.example.com`); it does not execute chaincode.
- **Channel** — a private ledger shared by chosen organizations; here `mychannel`.
- **Chaincode (smart contract)** — business logic installed on peers; here the `asset-transfer-basic` JavaScript chaincode, which manages assets with the fields ID, Color, Size, Owner and AppraisedValue.
- **Membership Service Provider (MSP)** and **TLS certificates** — identities and secured communication; the `CORE_PEER_*` variables select which identity the `peer` CLI acts as.

Fabric transactions follow *execute-order-validate*: a client sends a proposal to endorsing peers, which simulate it and sign the result; the client submits the endorsed transaction to the orderer, which puts it in a block; peers then validate and commit it. This is why `peer chaincode invoke` must name the orderer and the endorsing peers of **both** organizations (the chaincode's endorsement policy), whereas `peer chaincode query` is read-only and only talks to the local peer.

Tools used: Hyperledger Fabric 2.5.16 binaries and Docker images, `fabric-samples` `test-network` (2 peer organizations, 1 orderer, channel `mychannel`), the `asset-transfer-basic` JavaScript chaincode, Docker (OrbStack) on macOS. The manual's Windows Subsystem for Linux + Ubuntu environment was not used; the same commands were run natively on macOS through `run_fabric.sh`, with the elided `peer chaincode invoke ...` flags written out in full.

Program

Commands from the manual (steps 6 to 15), run in one script, `run_fabric.sh`:

```
#!/bin/bash
# Experiment 10: Hyperledger Fabric test-network + asset-
        transfer-basic chaincode (steps 6-15 of the manual)
set -x
export PATH=$PWD/fabric-samples/bin:$PATH
export FABRIC_CFG_PATH=$PWD/fabric-samples/config
peer version
cd fabric-samples/test-network
./network.sh down
./network.sh up createChannel
./network.sh deployCC -ccn basic -ccp ../asset-transfer-
        basic/chaincode-javascript -ccl javascript

export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID=Org1MSP
export
        CORE_PEER_TLS_ROOTCERT_FILE=$PWD/organizations/peerOrga
        nizations/org1.example.com/peers/peer0.org1.example.com
        /tls/ca.crt
export
        CORE_PEER_MSPCONFIGPATH=$PWD/organizations/peerOrganiza
        tions/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051
ORDERER_CA=$PWD/organizations/ordererOrganizations/example.com/
        orderers/orderer.example.com/msp/tlscacerts/tlsca.examp
        le.com-cert.pem
ORG2_TLS=$PWD/organizations/peerOrganizations/org2.example.com/
        peers/peer0.org2.example.com/tls/ca.crt
INVOKE=(peer chaincode invoke -o localhost:7050 --
        ordererTLSHostnameOverride orderer.example.com --tls --
        cafile "$ORDERER_CA" -C mychannel -n basic
  --peerAddresses localhost:7051 --tlsRootCertFiles
        "$CORE_PEER_TLS_ROOTCERT_FILE" --peerAddresses
        localhost:9051 --tlsRootCertFiles "$ORG2_TLS")

"${INVOKE[@]}" -c '{"function":"InitLedger","Args":[]}'
sleep 3
peer chaincode query -C mychannel -n basic -c '{"Args":
        ["GetAllAssets"]}'
peer chaincode query -C mychannel -n basic -c '{"Args":
        ["ReadAsset","asset5"]}'
"${INVOKE[@]}" -c '{"function":"CreateAsset","Args":
        ["asset7","green","12","Yash","950"]}'
sleep 3
peer chaincode query -C mychannel -n basic -c '{"Args":
        ["ReadAsset","asset7"]}'
"${INVOKE[@]}" -c '{"function":"TransferAsset","Args":
        ["asset7","Tomoko"]}'
sleep 3
peer chaincode query -C mychannel -n basic -c '{"Args":
        ["ReadAsset","asset7"]}'
peer chaincode query -C mychannel -n basic -c '{"Args":
        ["AssetExists","asset99"]}'
docker ps --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}'
./network.sh down
```

Output

```
test-network $ peer version
peer:
 Version: v2.5.16
 Commit SHA: f871cf9
 Go version: go1.26.4
 OS/Arch: darwin/arm64
 Chaincode:
  Base Docker Label: org.hyperledger.fabric
  Docker Namespace: hyperledger

test-network $
```

The Fabric CLI is on the `PATH` with `FABRIC_CFG_PATH` set (manual step 6); `peer version` reports v2.5.16 on darwin/arm64.

```
Fetching the most recent configuration block for the channel
++ peer channel fetch config /Users/anonjr/Documents/College/BC/Experiment_10_Output/fabric-samples/test-network/channel-artifacts/config_block.pb -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com -c mychannel --tls --cafile /Users/anonjr/Documents/College/BC/Experiment_10_Output/fabric-samples/test-network/organizations/ordererOrganizations/example.com/tlsca/tlsca.example.com-cert.pem
2026-10-21 21:40:27.943 IST 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
2026-10-21 21:40:27.944 IST 0002 INFO [cli.common] readBlock -> Received block: 1
2026-10-21 21:40:27.945 IST 0003 INFO [channelCmd] fetch -> Retrieving last config block: 1
2026-10-21 21:40:27.945 IST 0004 INFO [cli.common] readBlock -> Received block: 1
Decoding config block to JSON and isolating config to /Users/anonjr/Documents/College/BC/Experiment_10_Output/fabric-samples/test-network/channel-artifacts/Org2MSPconfig.json
++ configtxlator proto_decode --input /Users/anonjr/Documents/College/BC/Experiment_10_Output/fabric-samples/test-network/channel-artifacts/config_block.pb --type common.Block --output /Users/anonjr/Documents/College/BC/Experiment_10_Output/fabric-samples/test-network/channel-artifacts/config_block.json
++ jq '.data.data[0].payload.data.config' /Users/anonjr/Documents/College/BC/Experiment_10_Output/fabric-samples/test-network/channel-artifacts/config_block.json
++ res=0
Generating anchor peer update transaction for Org2 on channel mychannel
++ jq '.channel_group.groups.Application.groups.Org2MSP.values += {"AnchorPeers":{"mod_policy": "Admins","value":{"anchor_peers": [{"host": "peer0.org2.example.com","port": 9051}]},"version": "0"}}' /Users/anonjr/Documents/College/BC/Experiment_10_Output/fabric-samples/test-network/channel-artifacts/Org2MSPconfig.json
++ res=0
++ configtxlator proto_encode --input /Users/anonjr/Documents/College/BC/Experiment_10_Output/fabric-samples/test-network/channel-artifacts/Org2MSPconfig.json --type common.Config --output /Users/anonjr/Documents/College/BC/Experiment_10_Output/fabric-samples/test-network/channel-artifacts/original_config.pb
++ configtxlator proto_encode --input /Users/anonjr/Documents/College/BC/Experiment_10_Output/fabric-samples/test-network/channel-artifacts/Org2MSPmodified_config.json --type common.Config --output /Users/anonjr/Documents/College/BC/Experiment_10_Output/fabric-samples/test-network/channel-artifacts/modified_config.pb
++ configtxlator compute_update --channel_id mychannel --original /Users/anonjr/Documents/College/BC/Experiment_10_Output/fabric-samples/test-network/channel-artifacts/original_config.pb --updated /Users/anonjr/Documents/College/BC/Experiment_10_Output/fabric-samples/test-network/channel-artifacts/modified_config.pb --output /Users/anonjr/Documents/College/BC/Experiment_10_Output/fabric-samples/test-network/channel-artifacts/config_update.pb
++ configtxlator proto_decode --input /Users/anonjr/Documents/College/BC/Experiment_10_Output/fabric-samples/test-network/channel-artifacts/config_update.pb --type common.ConfigUpdate --output /Users/anonjr/Documents/College/BC/Experiment_10_Output/fabric-samples/test-network/channel-artifacts/config_update.json
++ jq .
+++ cat /Users/anonjr/Documents/College/BC/Experiment_10_Output/fabric-samples/test-network/channel-artifacts/config_update.json
++ echo '{"payload":{"header":{"channel_header":{"channel_id":"mychannel", "type":2}},"data":{"config_update":{' '"channel_id":' '"mychannel",' '"isolated_data":' '{},' '"read_set":' '{' '"groups":' '{' '"Application":' '{' '"groups":' '{' '"Org2MSP":' '{' '"groups":' '{},' '"mod_policy":' '"",' '"policies":' '{' '"Admins":' '{' '"mod_policy":' '"",' '"policy":' null, '"version":' '"0"' '},' '"Endorsement":' '{' '"mod_policy":' '"",' '"policy":' null, '"version":' '"0"' '},' '"Readers":' '{' '"mod_policy":' '"",' '"policy":' null, '"version":' '"0"' '},' '"Writers":' '{' '"mod_policy":' '"",' '"policy":' null, '"version":' '"0"' '}' '},' '"values":' '{' '"MSP":' '{' '"mod_policy":' '"",' '"value":' null, '"version":' '"0"' '}' '},' '"version":' '"0"' '}' '},' '"mod_policy":' '"",' '"policies":' '{},' '"values":' '{},' '"version":' '"0"' '}' '},' '"mod_policy":' '"",' '"policies":' '{},' '"values":' '{},' '"version":' '"0"' '},' '"write_set":' '{' '"groups":' '{' '"Application":' '{' '"groups":' '{' '"Org2MSP":' '{' '"groups":' '{},' '"mod_policy":' '"Admins",' '"policies":' '{' '"Admins":' '{' '"mod_policy":' '"",' '"policy":' null, '"version":' '"0"' '},' '"Endorsement":' '{' '"mod_policy":' '"",' '"policy":' null, '"version":' '"0"' '},' '"Readers":' '{' '"mod_policy":' '"",' '"policy":' null, '"version":' '"0"' '},' '"Writers":' '{' '"mod_policy":' '"",' '"policy":' null, '"version":' '"0"' '}' '},' '"values":' '{' '"AnchorPeers":' '{' '"mod_policy":' '"Admins",' '"value":' '{' '"anchor_peers":' '[' '{' '"host":' '"peer0.org2.example.com",' '"port":' 9051 '}' ']' '},' '"version":' '"0"' '},' '"MSP":' '{' '"mod_policy":' '"",' '"value":' null, '"version":' '"0"' '}' '},' '"version":' '"1"' '}' '},' '"mod_policy":' '"",' '"policies":' '{},' '"values":' '{},' '"version":' '"0"' '}' '},' '"mod_policy":' '"",' '"policies":' '{},' '"values":' '{},' '"version":' '"0"' '}' '}}}}'
++ configtxlator proto_encode --input /Users/anonjr/Documents/College/BC/Experiment_10_Output/fabric-samples/test-network/channel-artifacts/config_update_in_envelope.json --type common.Envelope --output /Users/anonjr/Documents/College/BC/Experiment_10_Output/fabric-samples/test-network/channel-artifacts/Org2MSPanchors.tx
2026-10-21 21:40:28.068 IST 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
2026-10-21 21:40:28.094 IST 0002 INFO [channelCmd] update -> Successfully submitted channel update
Anchor peer set for org 'Org2MSP' on channel 'mychannel'
Channel 'mychannel' joined
test-network $
```

Step 7: `./network.sh up createChannel` generates the certificates, starts the containers, creates `mychannel`, joins both peers and sets the anchor peers. The last lines of its output are shown: “Anchor peer set for org ‘Org2MSP’” and “Channel ‘mychannel’ joined”.

```
test-network $ docker ps --filter name=example.com --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}'
NAMES                   IMAGE                               STATUS
peer0.org2.example.com   hyperledger/fabric-peer:latest      Up 29 seconds
peer0.org1.example.com   hyperledger/fabric-peer:latest      Up 29 seconds
orderer.example.com      hyperledger/fabric-orderer:latest   Up 29 seconds
test-network $
```

`docker ps` *shows the two peers (`peer0.org1`, `peer0.org2`) and the orderer running.*

```
Checking the commit readiness of the chaincode definition on peer0.org1 on channel 'mychannel'...
Attempting to check the commit readiness of the chaincode definition on peer0.org1, Retry after 3 seconds.
+ peer lifecycle chaincode checkcommitreadiness --channelID mychannel --name basic --version 1.0 --sequence 1 --output json
+ res=0
{
        "approvals": {
                "Org1MSP": true,
                "Org2MSP": true
        }
}
Checking the commit readiness of the chaincode definition successful on peer0.org1 on channel 'mychannel'
Using organization 2
Checking the commit readiness of the chaincode definition on peer0.org2 on channel 'mychannel'...
Attempting to check the commit readiness of the chaincode definition on peer0.org2, Retry after 3 seconds.
+ peer lifecycle chaincode checkcommitreadiness --channelID mychannel --name basic --version 1.0 --sequence 1 --output json
+ res=0
{
        "approvals": {
                "Org1MSP": true,
                "Org2MSP": true
        }
}
Checking the commit readiness of the chaincode definition successful on peer0.org2 on channel 'mychannel'
Using organization 1
Using organization 2
+ peer lifecycle chaincode commit -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile /Users/anonjr/Documents/College/BC/E
xperiment_10_Output/fabric-samples/test-network/organizations/ordererOrganizations/example.com/tlsca/tlsca.example.com-cert.pem --channelID mychannel --
name basic --peerAddresses localhost:7051 --tlsRootCertFiles /Users/anonjr/Documents/College/BC/Experiment_10_Output/fabric-samples/test-network/organiz
ations/peerOrganizations/org1.example.com/tlsca/tlsca.org1.example.com-cert.pem --peerAddresses localhost:9051 --tlsRootCertFiles /Users/anonjr/Document
s/College/BC/Experiment_10_Output/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/tlsca/tlsca.org2.example.com-cert.pem --v
ersion 1.0 --sequence 1
+ res=0
2026-10-21 21:41:17.227 IST 0001 INFO [chaincodeCmd] ClientWait -> txid [1d6fede129ff4a59c27d3975a1bec5dd606e8c81ced4dccd99f4db370b3d3090] committed wit
h status (VALID) at localhost:7051
2026-10-21 21:41:17.228 IST 0002 INFO [chaincodeCmd] ClientWait -> txid [1d6fede129ff4a59c27d3975a1bec5dd606e8c81ced4dccd99f4db370b3d3090] committed wit
h status (VALID) at localhost:9051
Chaincode definition committed on channel 'mychannel'
Using organization 1
Querying chaincode definition on peer0.org1 on channel 'mychannel'...
Attempting to Query committed status on peer0.org1, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org1 on channel 'mychannel'
Using organization 2
Querying chaincode definition on peer0.org2 on channel 'mychannel'...
Attempting to Query committed status on peer0.org2, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org2 on channel 'mychannel'
Chaincode initialization is not required
test-network $
```

Step 8: `./network.sh deployCC` packages, installs, approves and commits the `basic` chaincode. Both organizations show `true` in the commit-readiness check, and "Committed chaincode definition for chaincode 'basic' on channel 'mychannel'" confirms Version 1.0, Sequence 1, Approvals [Org1MSP: true, Org2MSP: true].

```
test-network $ env | grep CORE_PEER | sed "s#$PWD#.#"
CORE_PEER_TLS_ENABLED=true
CORE_PEER_LOCALMSPID=Org1MSP
CORE_PEER_ADDRESS=localhost:7051
CORE_PEER_TLS_ROOTCERT_FILE=./organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
CORE_PEER_MSPCONFIGPATH=./organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
test-network $
```

The `CORE_PEER_*` variables make the `peer` CLI act as the Org1 admin against `peer0.org1` with TLS enabled.

```
test-network $ peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic
 --peerAddresses localhost:7051 --tlsRootCertFiles $CORE_PEER_TLS_ROOTCERT_FILE --peerAddresses localhost:9051 --tlsRootCertFiles $ORG2_TLS -c '{"functi
on":"InitLedger","Args":[]}'
2026-10-21 21:41:58.684 IST 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
test-network $ peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"},{"AppraisedValue":400,"Color":"red","ID":"asset2","Owne
r":"Brad","Size":5,"docType":"asset"},{"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"},{"AppraisedValu
e":600,"Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"},{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","
Size":15,"docType":"asset"},{"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]
test-network $ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'
{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}
test-network $
```

Steps 10 to 12: the `InitLedger` invoke (naming the orderer and the endorsing peers of both organizations) returns "Chaincode invoke successful. result: status:200". `GetAllAssets` lists the six sample assets and `ReadAsset` returns `asset5`.

```
test-network $ peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic
 --peerAddresses localhost:7051 --tlsRootCertFiles $CORE_PEER_TLS_ROOTCERT_FILE --peerAddresses localhost:9051 --tlsRootCertFiles $ORG2_TLS -c '{"functi
on":"CreateAsset","Args":["asset7","green","12","Yash","950"]}'
2026-10-21 21:42:20.659 IST 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"{\"ID\":\"asset7
\",\"Color\":\"green\",\"Size\":12,\"Owner\":\"Yash\",\"AppraisedValue\":950}"
test-network $ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset7"]}'
{"AppraisedValue":950,"Color":"green","ID":"asset7","Owner":"Yash","Size":12}
test-network $
```

Step 13: `CreateAsset` adds `asset7` (green, size 12, owner Yash, value 950), and a `ReadAsset` query confirms it is on the ledger.

```
test-network $ peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic
 --peerAddresses localhost:7051 --tlsRootCertFiles $CORE_PEER_TLS_ROOTCERT_FILE --peerAddresses localhost:9051 --tlsRootCertFiles $ORG2_TLS -c '{"functi
on":"TransferAsset","Args":["asset7","Tomoko"]}'
2026-10-21 21:42:30.255 IST 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"Yash"
test-network $ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset7"]}'
{"AppraisedValue":950,"Color":"green","ID":"asset7","Owner":"Tomoko","Size":12}
test-network $ peer chaincode query -C mychannel -n basic -c '{"Args":["AssetExists","asset99"]}'
false
test-network $
```

Step 14: `TransferAsset` returns the previous owner ("Yash"). `ReadAsset` then shows the owner is Tomoko, and `AssetExists` for a non-existent ID returns `false`.



Step 15: before shutdown, `docker ps` lists the peers, the orderer and the two chaincode containers. `./network.sh down` removes them, along with the volumes and the generated chaincode images, and a final `docker ps` shows no Fabric containers left.

(The run log for an earlier pass of the same commands is saved as `fabric_run_clean.log`, produced by `run_fabric.sh`, both in this folder.)

Conclusion

This experiment implemented a Hyperledger Fabric network (two peer organizations, one orderer and a channel, `mychannel`) and deployed the `asset-transfer-basic` chaincode with both organizations approving the definition. Asset management was demonstrated end to end: the ledger was initialized with six assets, queried (all assets and one by ID), a new asset `asset7` was created with owner Yash, and ownership was then transferred to Tomoko, with a read-back after each transaction confirming the committed state. Read operations (`query`) worked directly against one peer, while writes (`invoke`) had to be endorsed by both organizations and ordered into a block, illustrating Fabric's execute-order-validate model and the difference from the public, Proof-of-Work chains of the earlier experiments. The run was done natively on macOS rather than in WSL/Ubuntu as the manual assumes, and used the standard sample asset case study with the asset owned by the author, not a separately designed case.